AI-Enhanced Incident Reporting & Resolution System
Project Report

SECJ3483 – Web Technology
Universiti Teknologi Malaysia
Individual Assignment

Prepared by: MUHAMMAD DINIE HAIKAL BIN WANNORHAIZY

Matric NO: A22MJ8030

Lecturer: DR. MUHAMMAD AIMAN

1. Introduction

The AI-Enhanced Incident Reporting & Resolution System was developed to assist DHL customer support and logistics operations in managing incidents more efficiently. In large logistics organizations, incidents such as delayed deliveries, damaged parcels, incorrect addresses, and customer complaints occur frequently every day. Traditionally, many of these reports are handled manually through emails, chat messages, phone calls, and handwritten notes, which often causes delays and inconsistencies in the workflow. This project focuses on building a centralized web-based system that allows users to report incidents, monitor progress, and generate management reports in a structured and organized environment. The system was designed using modern web technologies including HTML, CSS, JavaScript, JSON Server, and REST API integration. In addition, the project also introduces automation concepts through RPA integration, allowing repetitive tasks such as report handling, status updates, and monitoring processes to be automated efficiently. The main objective of this project is to improve operational efficiency, reduce manual workload, and provide a better incident management experience for both employees and administrators.

2. Project Objectives

The main objective of this project is to design and develop a functional incident reporting system that can help DHL operations manage logistics incidents effectively. The system aims to provide secure login functionality, allow users to create and manage incidents, support incident tracking, and generate reports for monitoring purposes. Another important objective is to implement CRUD operations through JSON API integration, ensuring that all data can be created, retrieved, updated, and deleted dynamically. The project also aims to demonstrate understanding of frontend and backend integration while applying responsive interface design principles. Furthermore, the project includes RPA automation concepts that simulate how repetitive operational tasks can be automated to improve productivity and reduce human errors within logistics operations.

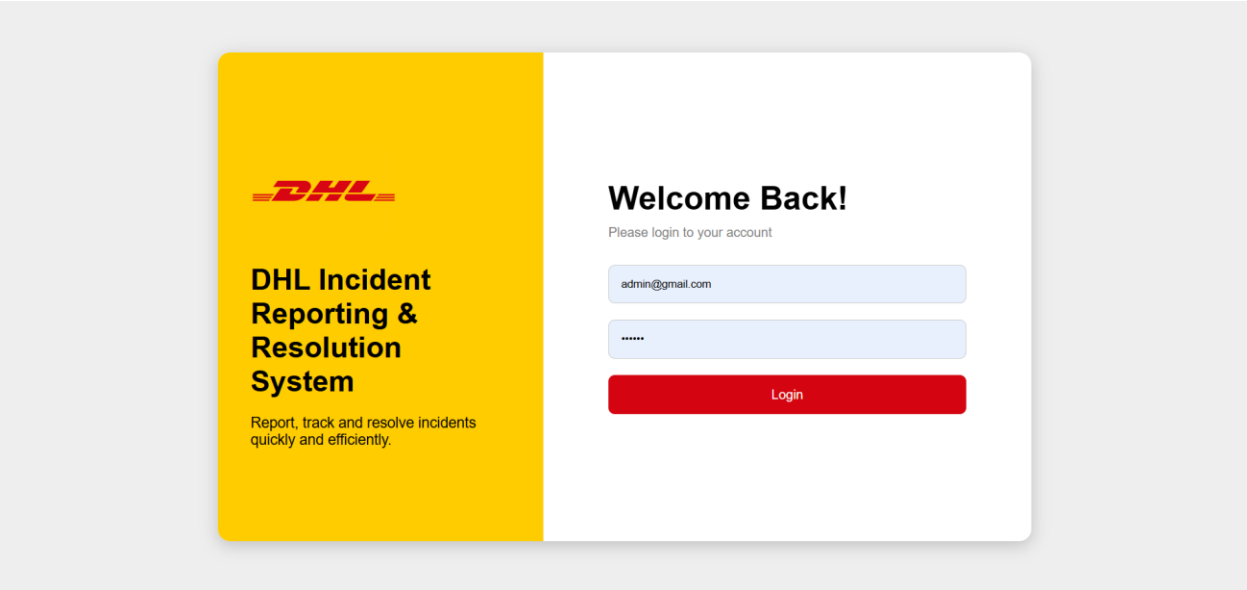
3. System Architecture

The system architecture consists of three main components, which are the frontend interface, backend data management, and automation workflow. The frontend was

developed using HTML, CSS, and JavaScript to create an interactive and user-friendly interface. The backend uses JSON Server as a mock API database that stores all incident information dynamically inside the db.json file. Communication between the frontend and backend is handled using REST API requests such as GET, POST, PUT, and DELETE. The automation layer represents the RPA process, where repetitive actions such as reading incident data, updating statuses, and generating summaries can be automated using UiPath workflows. This architecture ensures that the system remains organized, scalable, and suitable for real-world logistics incident management operations.

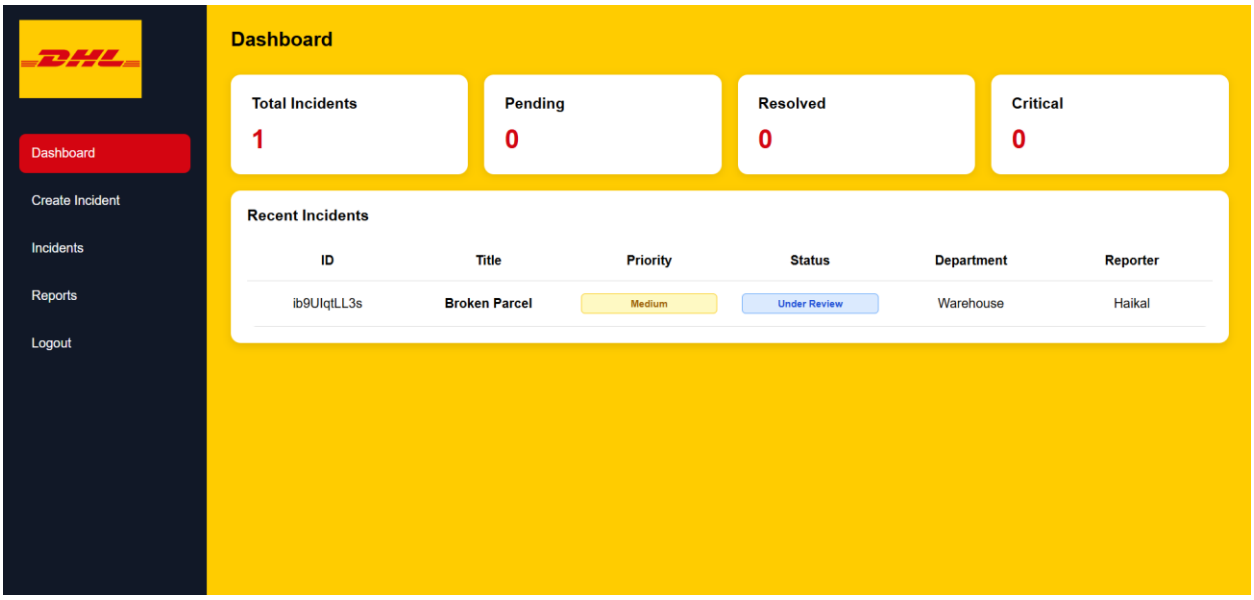
4. Login Module

The login module acts as the main security layer of the system by ensuring that only authorized users can access the incident management dashboard. Users are required to enter a valid email address and password before entering the system. The system validates the credentials using JavaScript logic and redirects successful users to the main dashboard page. If invalid credentials are entered, the system immediately displays an error message to prevent unauthorized access. The login page was designed with DHL branding elements such as the DHL logo and a professional layout to improve the overall user experience. This module ensures that all incident data and reporting functionalities remain protected and accessible only to authorized personnel.



5. Dashboard Module

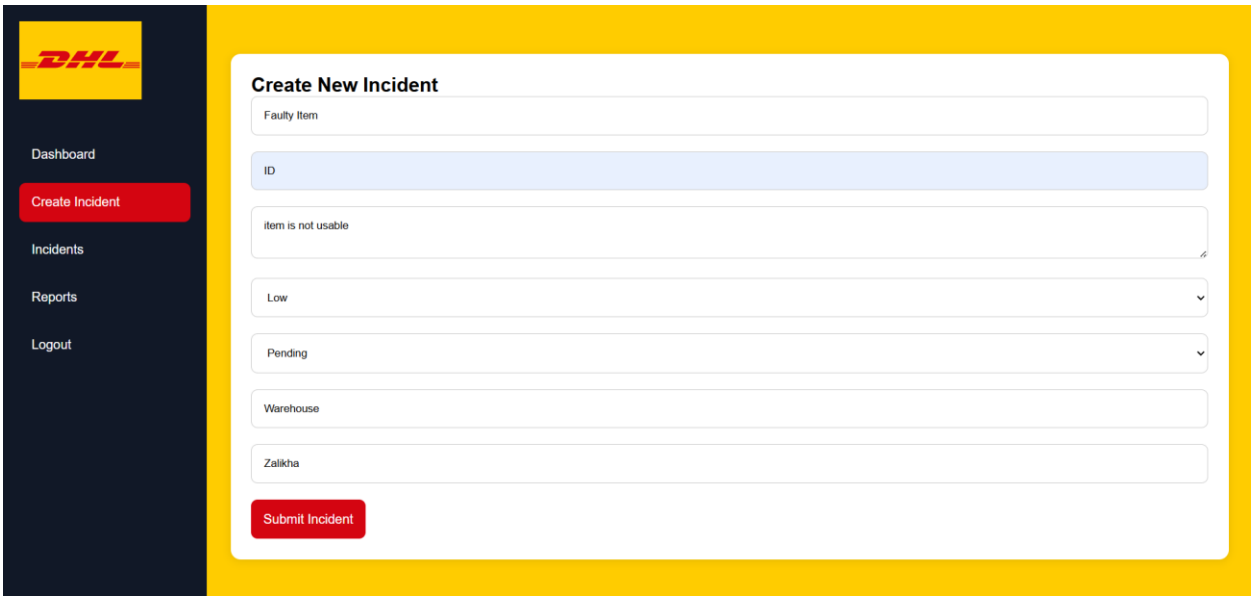
The dashboard module serves as the main monitoring center of the system. After successful login, users are redirected to the dashboard where they can view summarized information related to incidents. The dashboard displays important statistics such as the total number of incidents, pending incidents, resolved incidents, and critical incidents. These statistics provide quick visibility into operational performance and incident trends. The dashboard also contains a recent incidents table that displays the latest incident records together with important details such as incident ID, title, priority level, department, and status. The purpose of this module is to help users monitor incidents efficiently and make faster operational decisions.

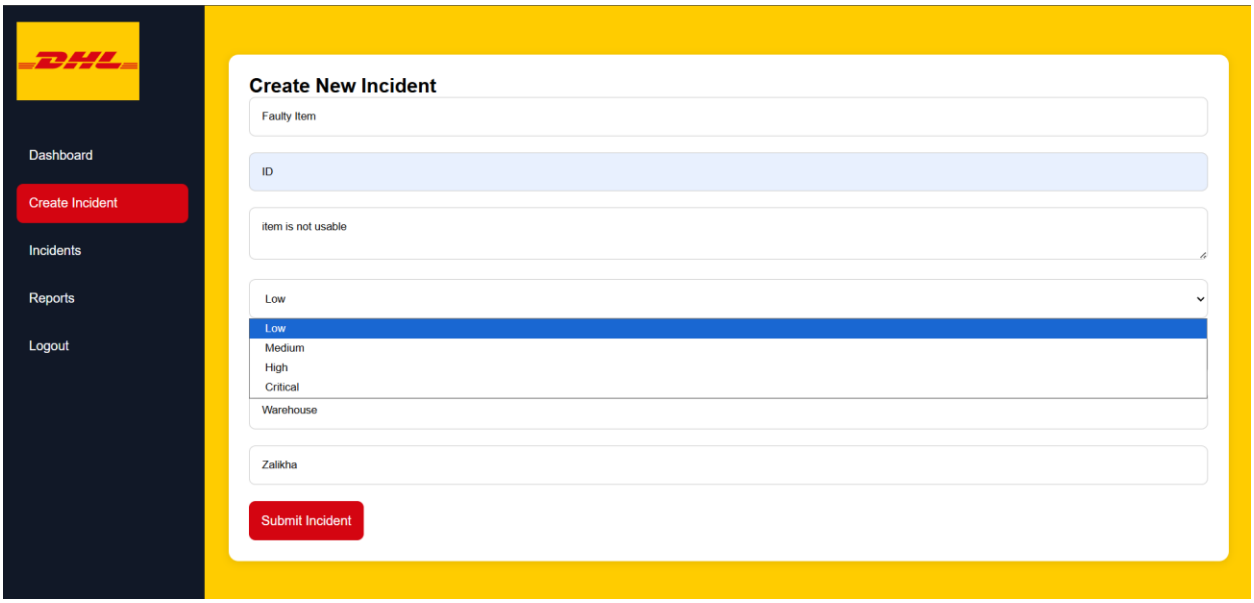


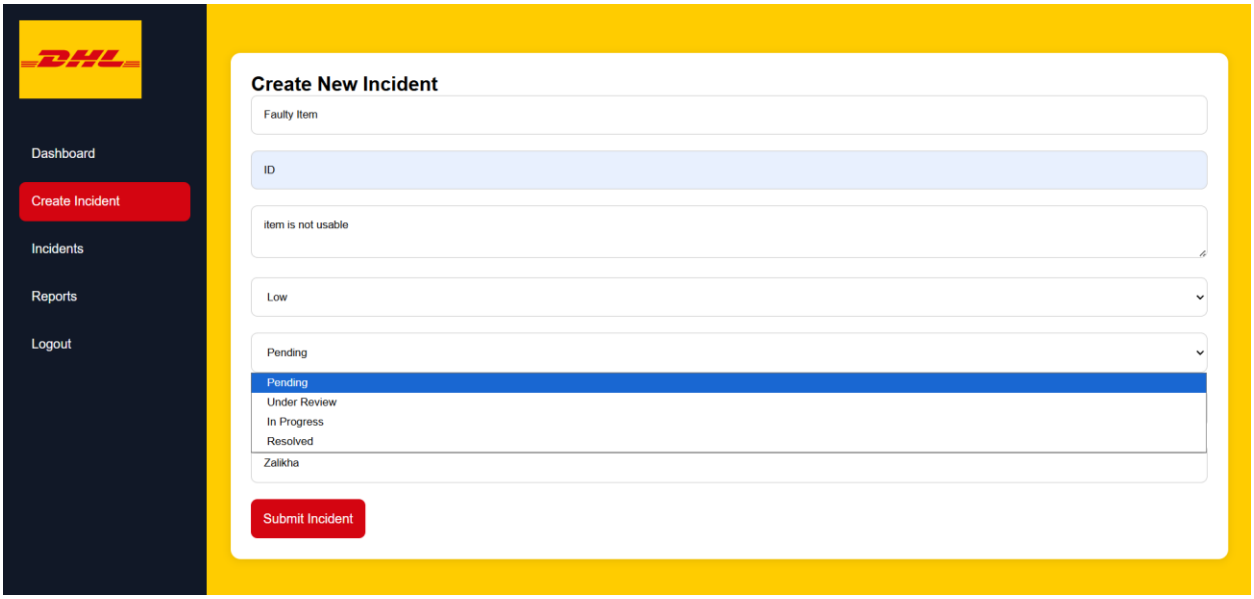
6. Create Incident Module

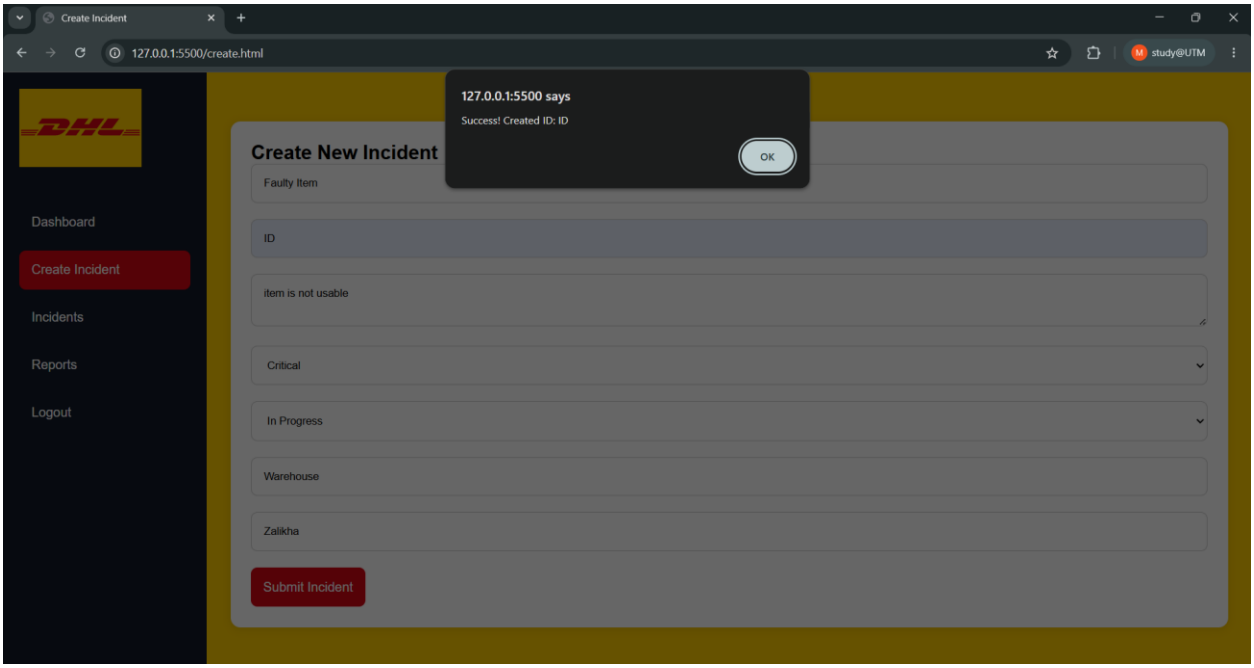
The Create Incident module allows users to submit new incidents into the system through a structured form interface. Users are required to provide important information such as incident ID, incident title, description, priority level, status, department name, and reporter details. The system validates the input fields to ensure that all necessary information is entered correctly before submission. JavaScript functions are used to handle form validation, data storage, and API communication. Once submitted, the incident information is stored dynamically inside the JSON database through a POST request. The module also includes draft-saving functionality using localStorage, allowing users to continue unfinished reports later

without losing information. This feature improves usability and prevents data loss during incident reporting.





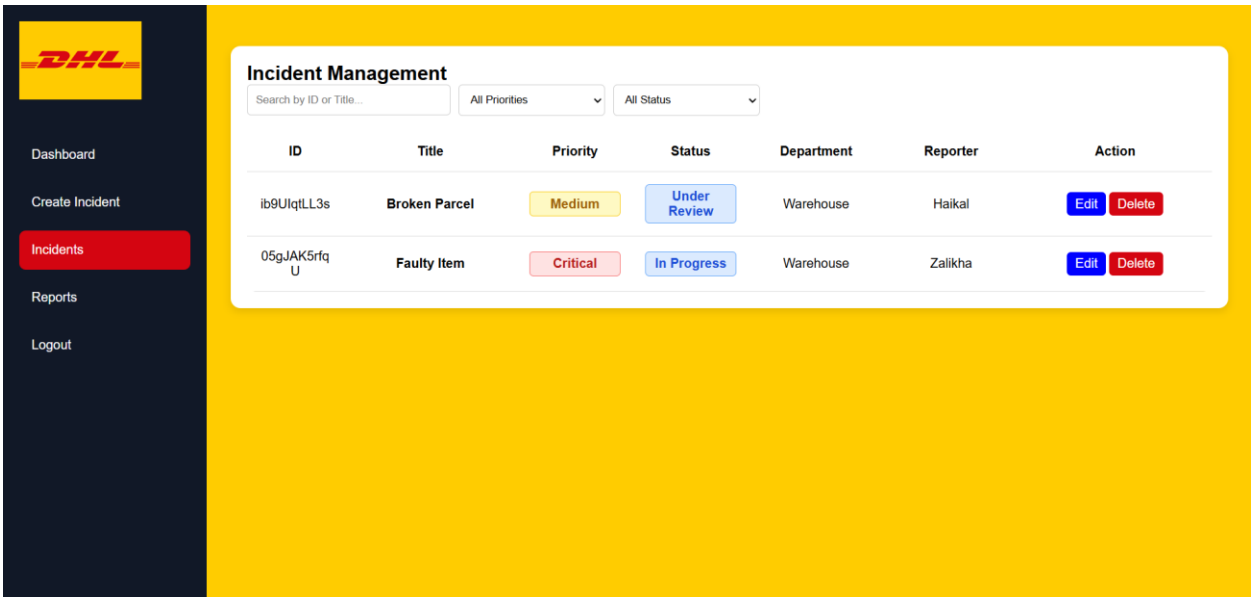


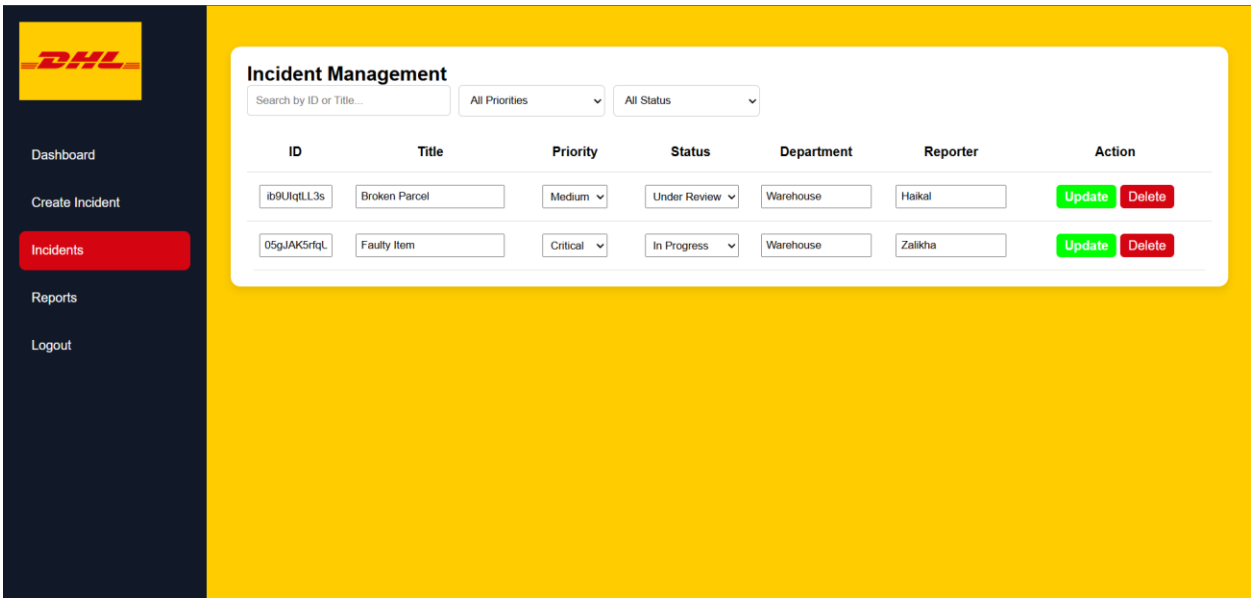


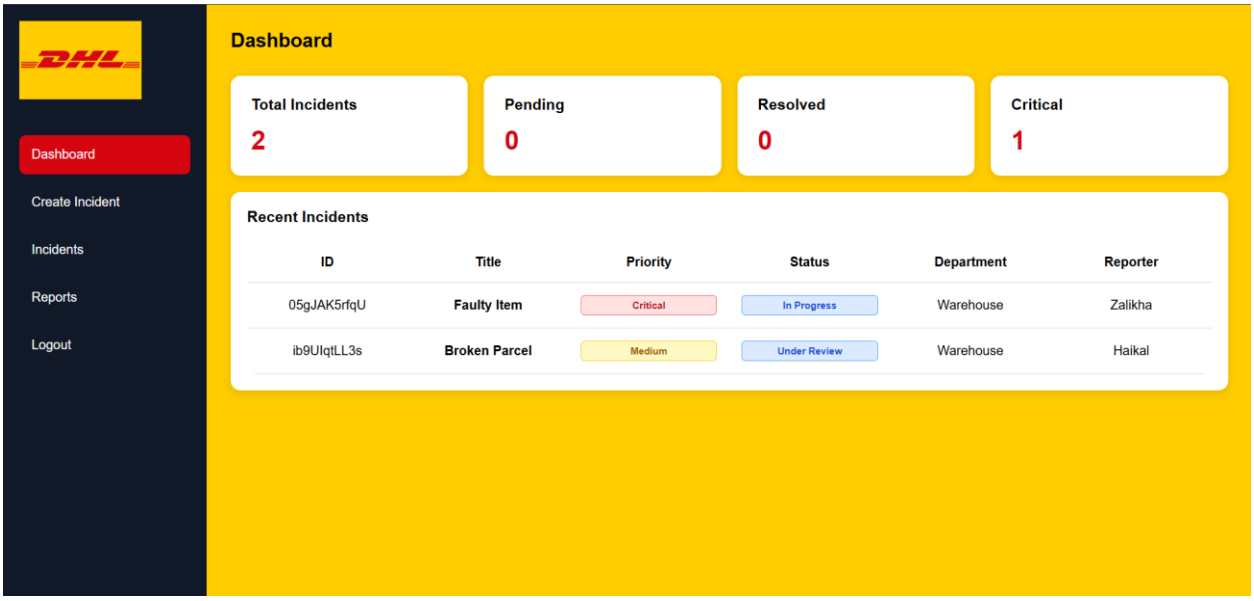
7. Incident Management Module

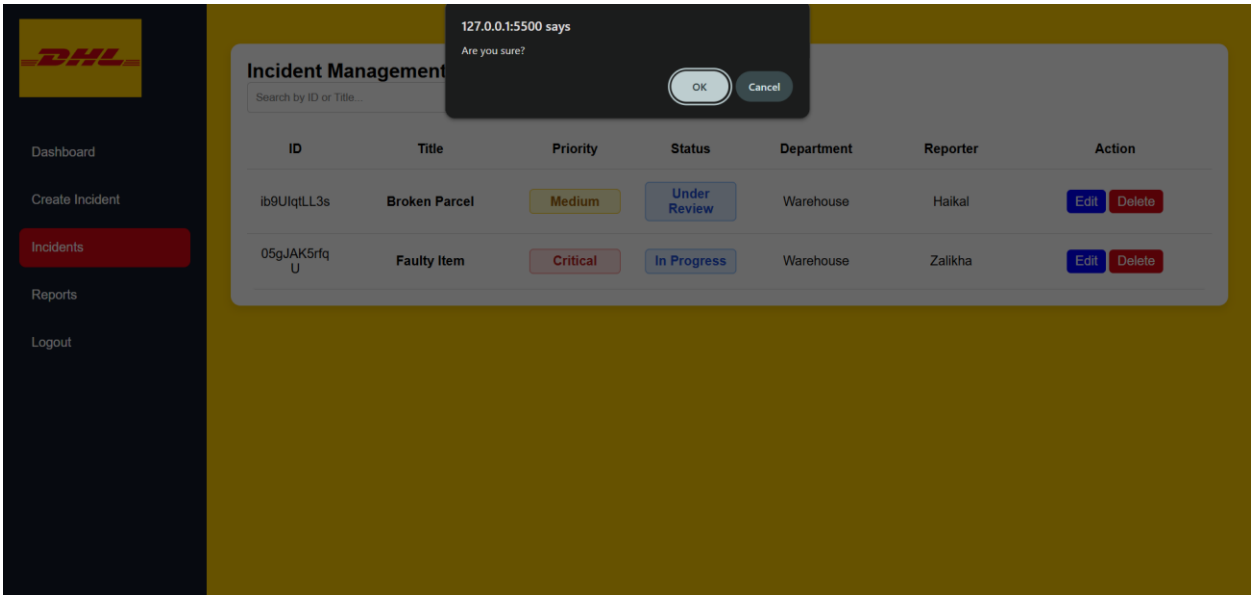
The Incident Management module allows users to manage all incidents stored inside the system. The module displays incidents in a table format with search and filtering functionalities. Users can search incidents using incident IDs or titles, making it easier to locate specific reports quickly. Filtering features are also provided to categorize incidents according to priority level and incident status. This module also allow user to delete incident if the issue is done and it will be updated in the

dashboard. This functionality improves data organization and allows users to monitor operational performance more effectively. The incident management page also demonstrates the implementation of CRUD operations, where users can retrieve and display data dynamically from the API server. This module plays an important role in maintaining incident records and ensuring that logistics operations remain organized.





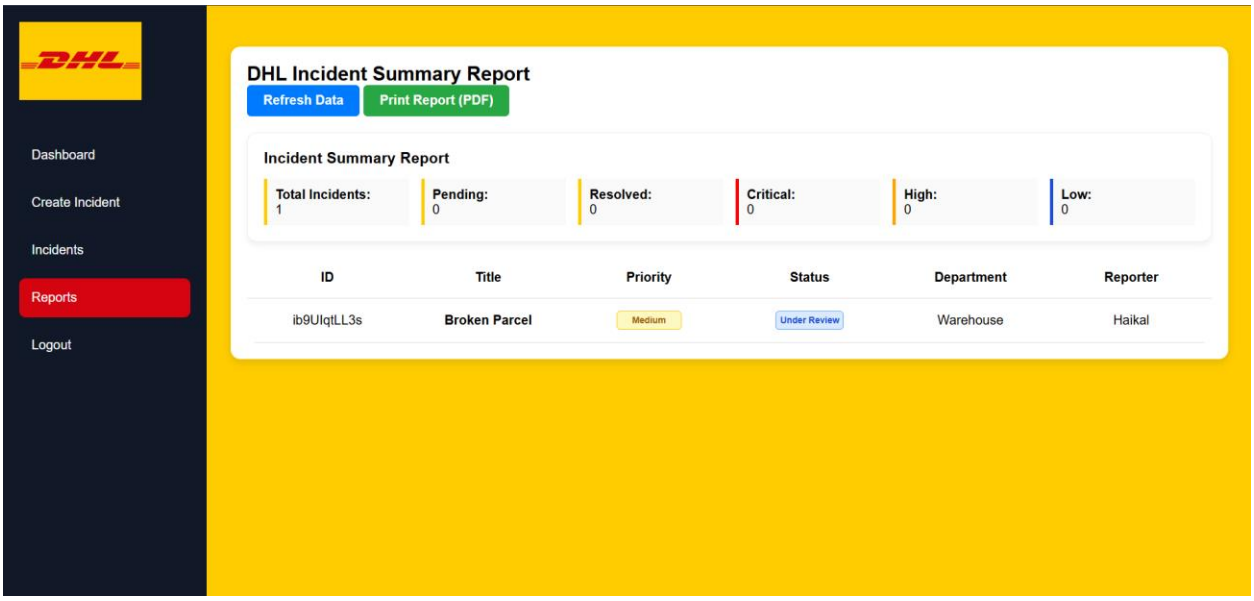


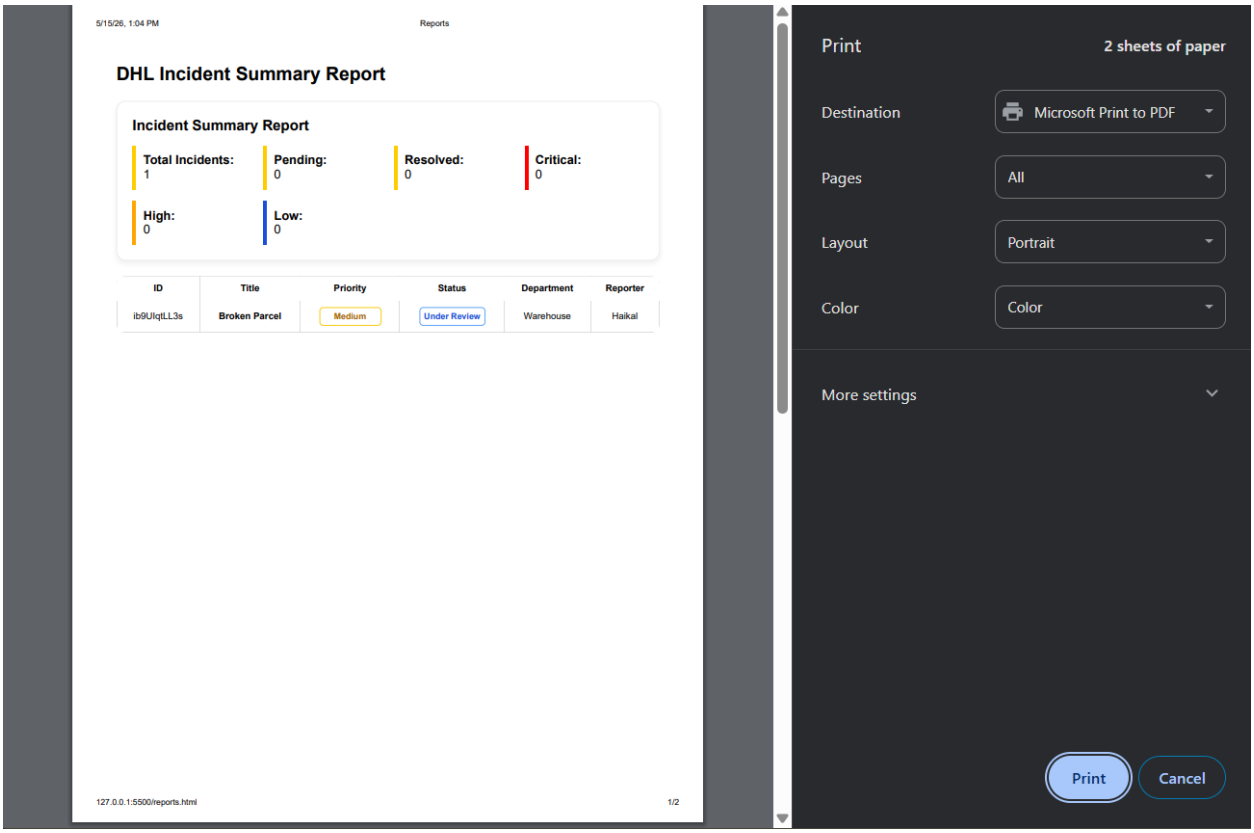


8. Reports Module

The Reports module was developed to help administrators and management teams monitor operational performance using summarized incident information. The system generates a structured report view containing important statistics and incident data. Users can review incident details and print reports for documentation purposes. The report page was designed with print optimization features that automatically remove unnecessary elements such as sidebars during printing. This ensures that printed reports appear professional and easy to read. The reports module improves

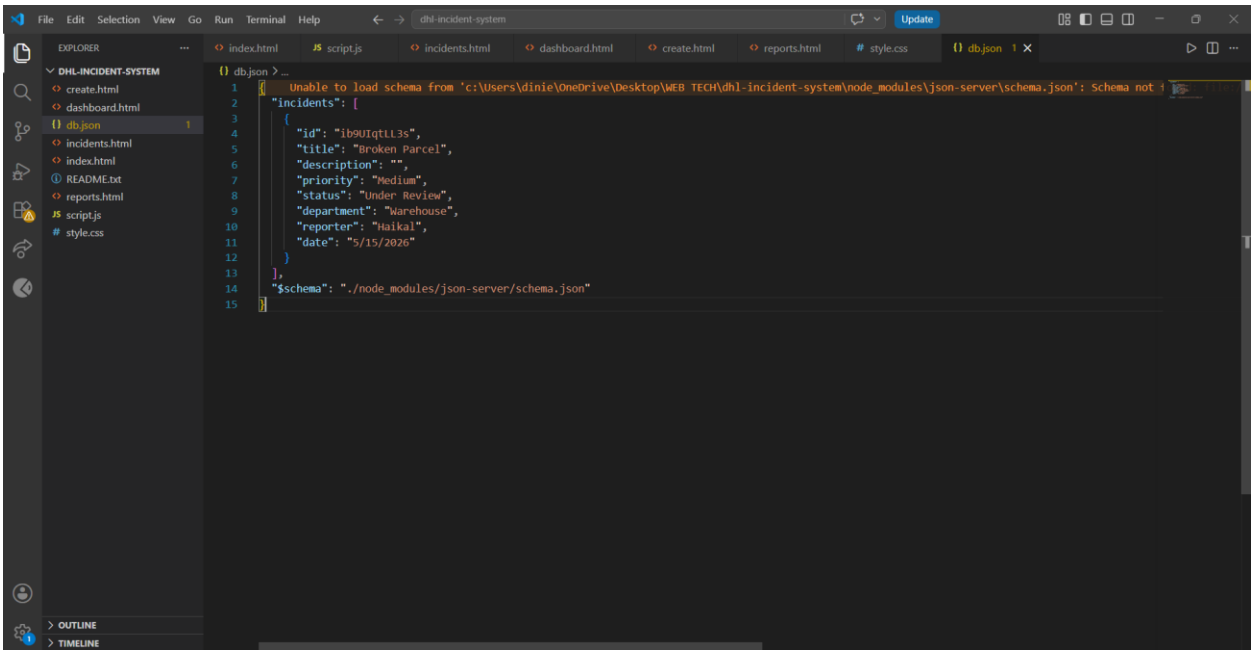
decision-making by providing clear visibility into incident trends, response performance, and operational challenges within the organization.

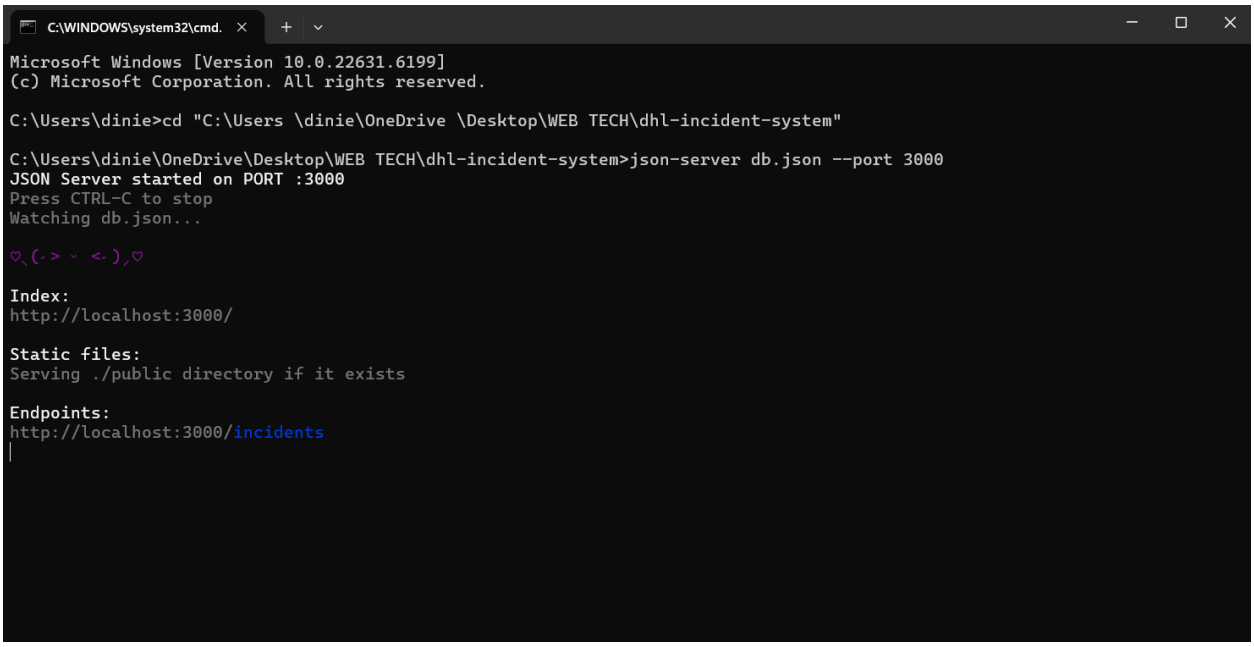




9. JSON API Integration

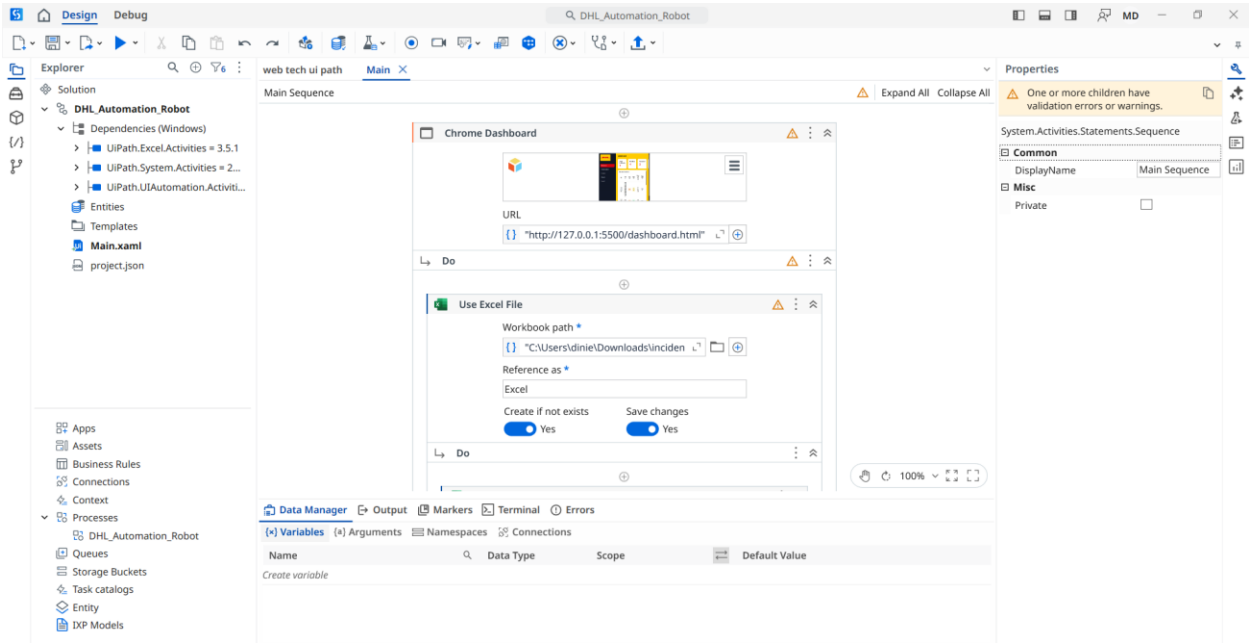
The system uses JSON Server as a mock backend API to manage incident data dynamically. The db.json file acts as the database storage where all incident information is saved. The application communicates with the backend using REST API methods including GET, POST, PUT, and DELETE operations. GET requests are used to retrieve incident records from the database, while POST requests are used to create new incidents. PUT and PATCH requests are used to update incident information, and DELETE requests can remove unnecessary records. This implementation demonstrates understanding of API integration and modern web application development concepts. Using JSON API integration also ensures that the frontend remains dynamic and scalable.

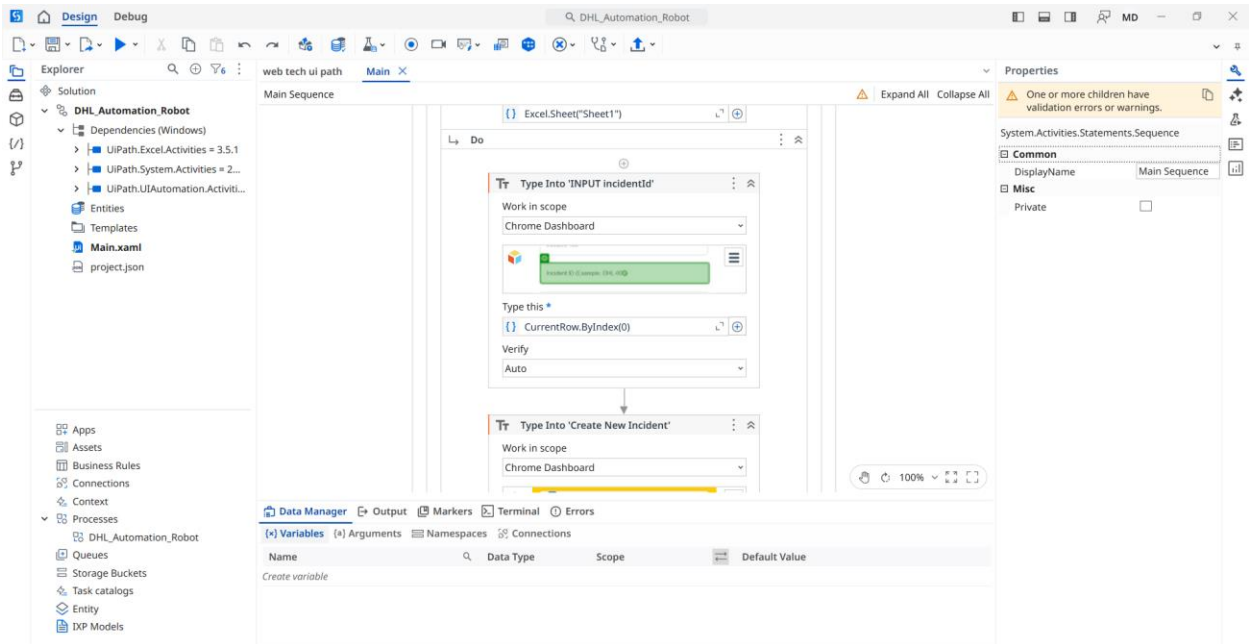


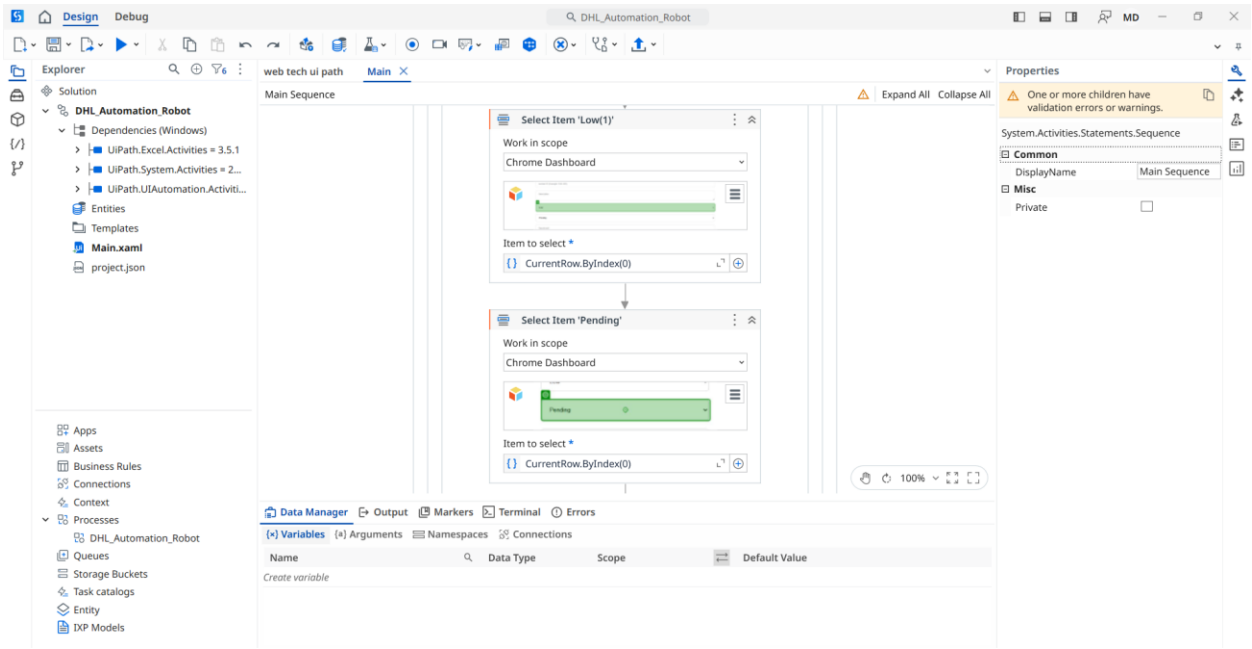


10. RPA Workflow and Automation

The RPA component of this project focuses on automating repetitive incident management tasks. Using UiPath, the workflow can automatically read incoming incident information, validate data, and update records inside the system. The automation process may include duplicate checking, automatic status updates, report generation, and summary email notifications. Error handling mechanisms such as Try/Catch activities and screenshot capturing are implemented to improve reliability during execution. The RPA workflow demonstrates how automation technology can improve operational efficiency within DHL logistics operations by reducing manual workload and minimizing processing errors.

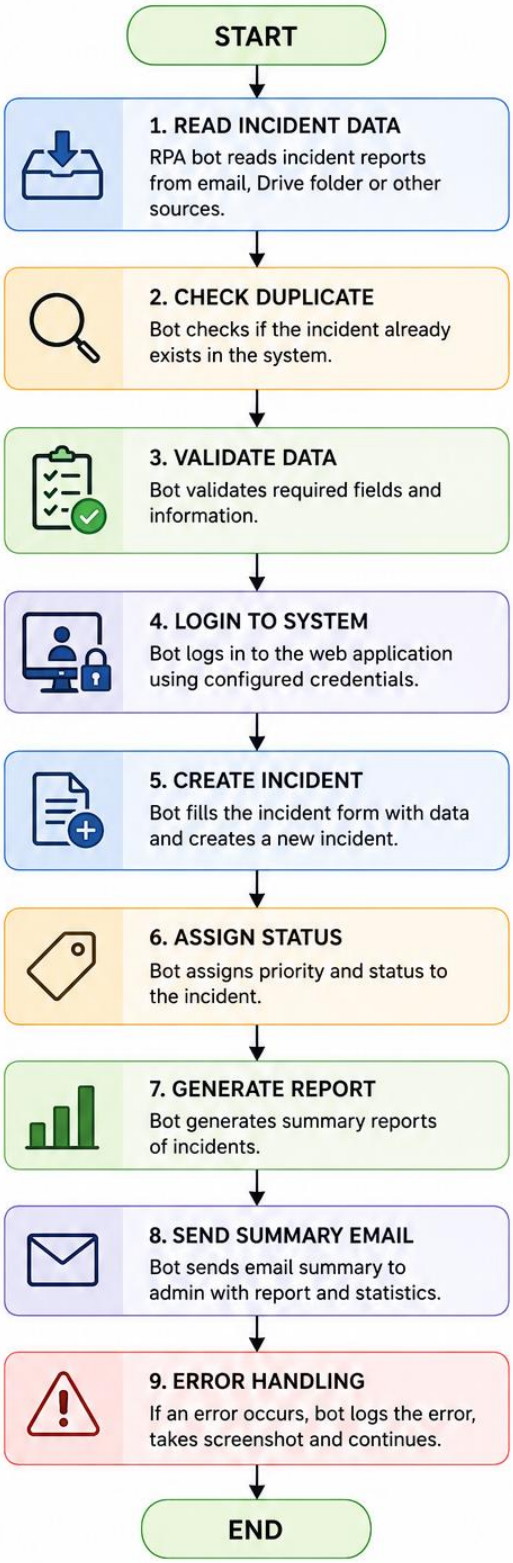

Design
Debug
DHL_Automation_Robot
MD
Explorer
Solution
DHL_Automation_Robot
Dependencies (Windows)
UiPath.Excel.Activities = 3.5.1
UiPath.System.Activities = 2...
UiPath.UIAutomation.Activiti...
Entities
Templates
Main.xaml
project.json
Apps
Assets
Business Rules
Connections
Context
Processes
DHL_Automation_Robot
Queues
Storage Buckets
Task catalogs
Entity
IXP Models
web tech ui path
Main
Main Sequence
Expand All
Collapse All
Chrome Dashboard
URL
"http://127.0.0.1:5500/dashboard.html"
Do
Use Excel File
Workbook path *
"C:\Users\dinie\Downloads\inciden
Reference as *
Excel
Create if not exists
Yes
Save changes
Yes
Do
100%
Data Manager
Output
Markers
Terminal
Errors
Variables
Arguments
Namespaces
Connections
Name
Data Type
Scope
Default Value
Create variable
Properties
One or more children have validation errors or warnings.
System.Activities.Statements.Sequence
Common
DisplayName
Main Sequence
Misc
Private


Design
Debug
DHL_Automation_Robot
MD
Explorer
Solution
DHL_Automation_Robot
Dependencies (Windows)
UiPath.Excel.Activities = 3.5.1
UiPath.System.Activities = 2...
UiPath.UIAutomation.Activiti...
Entities
Templates
Main.xaml
project.json
Apps
Assets
Business Rules
Connections
Context
Processes
DHL_Automation_Robot
Queues
Storage Buckets
Task catalogs
Entity
IXP Models
web tech ui path
Main
Main Sequence
Expand All
Collapse All
Excel.Sheet("Sheet1")
Do
Type Into 'INPUT incidentId'
Work in scope
Chrome Dashboard
Type this *
CurrentRow.ByIndex(0)
Verify
Auto
Type Into 'Create New Incident'
Work in scope
Chrome Dashboard
100%
Data Manager
Output
Markers
Terminal
Errors
Variables
Arguments
Namespaces
Connections
Name
Data Type
Scope
Default Value
Create variable
Properties
One or more children have validation errors or warnings.
System.Activities.Statements.Sequence
Common
DisplayName
Main Sequence
Misc
Private


Design
Debug
DHL_Automation_Robot
MD
Explorer
Solution
DHL_Automation_Robot
Dependencies (Windows)
UiPath.Excel.Activities = 3.5.1
UiPath.System.Activities = 2...
UiPath.UIAutomation.Activiti...
Entities
Templates
Main.xaml
project.json
Apps
Assets
Business Rules
Connections
Context
Processes
DHL_Automation_Robot
Queues
Storage Buckets
Task catalogs
Entity
IXP Models
web tech ui path
Main
Main Sequence
Expand All
Collapse All
Select Item 'Low(1)'
Work in scope
Chrome Dashboard
Item to select *
CurrentRow.ByIndex(0)
Select Item 'Pending'
Work in scope
Chrome Dashboard
Pending
Item to select *
CurrentRow.ByIndex(0)
100%
Data Manager
Output
Markers
Terminal
Errors
Variables
Arguments
Namespaces
Connections
Name
Data Type
Scope
Default Value
Create variable
Properties
One or more children have validation errors or warnings.
System.Activities.Statements.Sequence
Common
DisplayName
Main Sequence
Misc
Private

RPA WORKFLOW DIAGRAM

AI-Enhanced Incident Reporting & Resolution System



11. Challenges and Solutions

During the development process, several challenges were encountered. One of the main challenges involved integrating frontend functionalities with backend API communication. Managing asynchronous API requests and ensuring data consistency required careful implementation using JavaScript functions. Another challenge involved designing a clean and responsive user interface that remained user-friendly while displaying large amounts of incident information. The implementation of localStorage for draft saving also required additional testing to ensure that user data was stored and restored correctly. These challenges were solved through continuous testing, debugging, and improvement of the system structure. As a result, the final system was able to function correctly and meet the project requirements successfully.

12. Conclusion

In conclusion, the AI-Enhanced Incident Reporting & Resolution System successfully achieved the objectives defined for this project. The system provides a centralized platform for managing logistics incidents efficiently using modern web technologies and API integration. Features such as login authentication, incident creation, dashboard monitoring, incident filtering, report generation, and RPA workflow integration contribute to improving operational efficiency and incident handling processes. The project also demonstrates practical understanding of frontend development, backend integration, JSON data management, and automation concepts. Overall, this project provides valuable experience in developing real-world logistics management systems while preparing for future improvements and industry-level implementations.

13. Link:

Youtube Link for demo Video:
https://youtu.be/4IRWnOu94oo

Command Prompt Link:
C:\Users\dinie\OneDrive\Desktop\WEB TECH\WT_Assignment_MUHAMMADDINIEHAIKAL_A22MJ8030\dhl-incident-system